

PROMPT

BUILD A COMPLETE FULL-STACK AI PROJECT

PROJECT TITLE:

Config Drift Detector

OVERVIEW:

Build a complete AI-powered Config Drift Detection platform that compares intended configuration files against actual deployed configuration files, detects configuration drift, classifies risk severity, generates AI-powered impact analysis using Groq LLM, stores results in SQLite, visualizes drift trends, and exports PDF reports.

TECH STACK:

- * Frontend: Streamlit
- * Backend: Python
- * AI Model: Groq API (Llama 3 or latest supported model)
- * Database: SQLite
- * Config Formats: JSON and YAML
- * Visualization: Plotly
- * PDF Reports: ReportLab
- * Diff Engine: DeepDiff
- * Deployment Ready: GitHub

FUNCTIONAL REQUIREMENTS

1. DASHBOARD

Create a professional DevOps-style dashboard with:

- * Dark theme
- * Sidebar navigation
- * Upload Intended Configuration Folder
- * Upload Actual Configuration Folder

- * Load Demo Configurations button
- * Compare Configurations button
- * Scan History page
- * Analytics page
- * Report page

2. CONFIG FILE SUPPORT

Support:

- * JSON
- * YAML
- * Nested structures
- * Arrays
- * Objects

Automatically match files by filename.

Example:

intended/

app.yaml

db.json

actual/

app.yaml

db.json

3. DRIFT DETECTION ENGINE

Detect:

- * Missing Keys
- * Added Keys
- * Modified Values
- * Data Type Changes

Store:

- * File Name
- * Config Path
- * Expected Value
- * Actual Value
- * Drift Type

Use DeepDiff.

Example Output:

File: app.yaml

Path: replicas

Expected: 3

Actual: 1

Type: Modified Value

4. SEVERITY CLASSIFICATION ENGINE

Critical:

authentication

authorization

encryption

security

firewall

access_control

ssl

tls

api_keys

High:

database

replication

backup

load_balancer

network

storage

Medium:

logging

monitoring

caching

metrics

Low:

ui

theme

display

dashboard

If no keyword matches:

Severity = Medium

Display severity badges:

* Critical = Red

* High = Orange

* Medium = Yellow

* Low = Green

5. AI IMPACT ANALYSIS (GROQ)

For every drift call Groq API.

Prompt Structure:

Analyse the following configuration drift:

File:

Path:

Expected Value:

Actual Value:

Severity:

Generate:

1. Technical Impact
2. Business Impact
3. Security Impact
4. Risk Rating
5. Recommended Fix

Return structured JSON.

6. SQLITE DATABASE

Create tables:

comparison_history

id INTEGER PRIMARY KEY

timestamp DATETIME

file_name TEXT

config_path TEXT

drift_type TEXT

severity TEXT

intended_value TEXT

actual_value TEXT

ai_analysis

id INTEGER PRIMARY KEY

comparison_id INTEGER

technical_impact TEXT

business_impact TEXT

security_impact TEXT

risk_rating TEXT

recommendation TEXT

Store every scan.

7. ANALYTICS DASHBOARD

Show KPI Cards:

- * Total Files Checked
- * Total Drifts
- * Critical Drifts
- * High Severity Drifts
- * Last Scan Time

Charts:

- * Severity Distribution
- * Drift Type Distribution
- * Drift Trends Over Time
- * Top Affected Files

Use Plotly.

8. SEARCH AND FILTER

Allow filtering by:

- * File Name
- * Severity
- * Drift Type
- * Date Range

9. SCAN HISTORY

Display all historical scans from SQLite.

Columns:

- * Timestamp
- * File Name
- * Severity
- * Drift Count

Allow drill-down into results.

10. REPORT GENERATION

Generate PDF report containing:

Executive Summary

Scan Timestamp

Drift Statistics

Severity Breakdown

Detailed Drift Table

AI Impact Analysis

Recommendations

Include company-style formatting.

Provide:

Export PDF button

11. DEMO MODE

Load Sample Configurations button.

Automatically create:

intended_config.yaml

server:

replicas: 3

ssl: true

database:

backup: true

actual_config.yaml

server:

replicas: 1

ssl: false

database:

backup: false

Use these to demonstrate drift detection.

12. PROJECT STRUCTURE

config_drift_detector/

app.py

drift_detector.py

ai_analysis.py

database.py

report_generator.py

sample_data.py

utils.py

database.db

requirements.txt

README.md

13. CODE QUALITY

Requirements:

- * Modular architecture
- * Clean code
- * Error handling
- * Type hints
- * Logging
- * Comments
- * Reusable functions

14. OUTPUT

Generate complete source code for all files.

Provide:

- * app.py
- * drift_detector.py
- * ai_analysis.py
- * database.py
- * report_generator.py
- * sample_data.py
- * requirements.txt
- * README.md

The application must run locally with:

```
pip install -r requirements.txt
```

```
streamlit run app.py
```

Generate fully working code, not pseudocode.